

Лабораторна робота №5

з теми «Написання тестів для графічного інтерфейсу»

Виконала:

ст. гр. ІП-22-1

Чемна (Микитин) О. І.

Перевірив:

Храбатин Р. І.

Мета: Навчитися створювати UI тести, розібратися в існуючих інструментах для створення UI тестів та навчитися їх обирати.

Короткі теоретичні відомості:

UI-тестування використовується для перевірки графічного інтерфейсу вебдодатку та його поведінки під час взаємодії з користувачем. Таке тестування дозволяє перевірити, чи правильно відкриваються сторінки, чи відображаються потрібні елементи, чи працюють кнопки, меню, пошук, фільтри та інші частини інтерфейсу.

На відміну від ручної перевірки, автоматизовані UI-тести можуть виконувати однакові дії багато разів і швидко повідомляти про помилки.

Для створення UI-тестів у JavaScript можна використовувати різні інструменти, наприклад Puppeteer, Cypress, WebDriver або Playwright.

У цій роботі використовується Puppeteer, оскільки він дозволяє керувати браузером Chrome або Chromium, відкривати сторінки, знаходити елементи за селекторами, натискати кнопки та перевіряти результат дій.

Завдання:

Завдання перше:

Обрати ресурс над яким будете проводити дослідження та перевіряти коректність роботи UI.

Завдання друге:

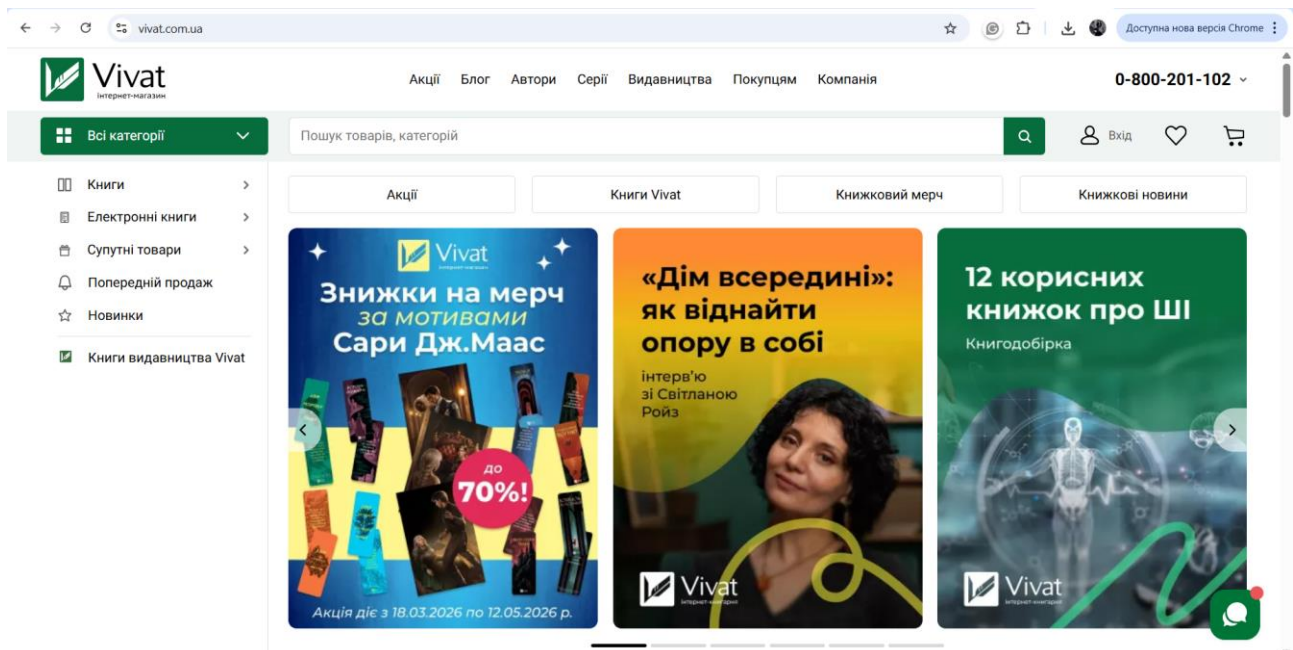
Написати короткі тест-кейси перевірки (можна взяти з лабораторних робіт «Якості та тестування програмного забезпечення»), написати щонайменше п'ять автоматизованих UI тестів.

Завдання третє:

Завантажити код на власний/навчальний репозиторій GitHub, або інший схожий ресурс та поділитися посиланням.

Виконання:

Для виконання роботи я обрала сайт Видавництва Віват:



Я обрала цей ресурс, оскільки він має зрозумілий графічний інтерфейс, каталог книжок, головну сторінку, пошук, меню та сторінки товарів. Такі елементи добре підходять для створення базових UI-тестів.

Спочатку я сформувала короткі тест-кейси для перевірки графічного інтерфейсу сайту.

№	Тест-кейс	Очікуваний результат
1	Відкрити головну сторінку сайту	Головна сторінка успішно відкривається
2	Перевірити заголовок сторінки	Заголовок сторінки містить інформацію про Vivat
3	Перевірити наявність навігації	На сторінці відображаються навігаційні елементи
4	Перевірити наявність посилань на сторінці	На сторінці є більше одного посилання
5	Перевірити перехід у каталог книг	Сторінка категорії книг відкривається
6	Перевірити наявність товарів у каталозі	У каталозі відображаються елементи книжок
7	Перевірити наявність поля пошуку	На сайті є елемент для пошуку

Після цього я створила окремий каталог для виконання лабораторної роботи:

```
oksanka@debian:~/Documents/automated_testing$ mkdir ui-testing
oksanka@debian:~/Documents/automated_testing$ cd ui-testing
```

Після створення каталогу я ініціалізувала Node.js-проект:

```
oksanka@debian:~/Documents/automated_testing/ui-testing$ npm init -y
Wrote to /home/oksanka/Documents/automated_testing/ui-testing/package.json:

{
  "name": "ui-testing",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

oksanka@debian:~/Documents/automated_testing/ui-testing$
```

Далі я встановила Jest для запуску тестів:

```
oksanka@debian:~/Documents/automated_testing/ui-testing$ npm install --save-dev jest
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@zsm.me
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@zsm.me

added 293 packages, and audited 294 packages in 14s

44 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
oksanka@debian:~/Documents/automated_testing/ui-testing$
```

Після цього я встановила Puppeteer для автоматизації роботи браузера:

```
oksanka@debian:~/Documents/automated_testing/ui-testing$ npm install --save-dev puppeteer

added 66 packages, and audited 360 packages in 2m

48 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
oksanka@debian:~/Documents/automated_testing/ui-testing$
```

Далі я відкрила файл package.json і змінила секцію scripts, щоб тести можна було запускати командою npm test:

```
> Debug
6 |   "scripts": {
7 |     "test": "jest"
8 |   },
```

Після налаштування проекту я створила файл для UI-тестів. У файл ui.test.js я додала тести для перевірки графічного інтерфейсу сайту Видавництва Віват:

package.json U

ui.test.js U X

ui.test.js > ...

```
1  const puppeteer = require("puppeteer");
2
3  jest.setTimeout(60000);
4
5  const BASE_URL = "https://vivat.com.ua/";
6  const BOOKS_URL = "https://vivat.com.ua/category/knyhy/";
7
8  describe("Vivat UI tests", () => {
9    let browser;
10    let page;
11
12    beforeEach(async () => {
13      browser = await puppeteer.launch({
14        headless: true,
15        args: ["--no-sandbox", "--disable-setuid-sandbox"],
16      });
17
18      page = await browser.newPage();
19
20      await page.setViewport({
21        width: 1366,
22        height: 768,
23      });
24    });
25
26    afterEach(async () => {
27      await browser.close();
28    });
29
```

```
29
30    test("should open the Vivat home page", async () => {
31      await page.goto(BASE_URL, {
32        waitUntil: "networkidle2",
33      });
34
35      const currentUrl = page.url();
36
37      expect(currentUrl).toContain("vivat.com.ua");
38    });
39
40    test("should display page title with Vivat information", async () => {
41      await page.goto(BASE_URL, {
42        waitUntil: "networkidle2",
43      });
44
45      const title = await page.title();
46
47      expect(title.toLowerCase()).toContain("vivat");
48    });
49
50    test("should contain navigation links on the home page", async () => {
51      await page.goto(BASE_URL, {
52        waitUntil: "networkidle2",
53      });
54
55      const linksCount = await page.$$eval("a", (links) => links.length);
56
```

```

56     expect(linksCount).toBeGreaterThan(0);
57   });
58
59
60   test("should open books category page", async () => {
61     await page.goto(BOOKS_URL, {
62       |   waitUntil: "networkidle2",
63     });
64
65     const currentUrl = page.url();
66
67     expect(currentUrl).toContain("/category/knyhy");
68   });
69
70   test("should display book-related content on books category page", async () => {
71     await page.goto(BOOKS_URL, {
72       |   waitUntil: "networkidle2",
73     });
74
75     const bodyText = await page.$eval("body", (body) => body.innerText);
76
77     expect(bodyText.toLowerCase()).toContain("книг");
78   });
79

```

```

79
80   test("should have multiple links on books category page", async () => {
81     await page.goto(BOOKS_URL, {
82       |   waitUntil: "networkidle2",
83     });
84
85     const linksCount = await page.$$eval("a", (links) => links.length);
86
87     expect(linksCount).toBeGreaterThan(5);
88   });
89
90   test("should have visible page content", async () => {
91     await page.goto(BASE_URL, {
92       |   waitUntil: "networkidle2",
93     });
94
95     const bodyTextLength = await page.$eval(
96       |   "body",
97       |   (body) => body.innerText.trim().length,
98     );
99
100    expect(bodyTextLength).toBeGreaterThan(100);
101  });
102 });

```

У цьому файлі я використала `beforeEach` для запуску браузера перед кожним тестом і `afterEach` для його закриття після виконання тесту. Це дозволило зробити тести незалежними один від одного.

Також я використала `jest.setTimeout(60000)`, оскільки UI-тести можуть виконуватися довше через завантаження сторінки, зображень та інших елементів інтерфейсу.

Після написання тестів я запустила їх у терміналі командою:

```

• oksanka@debian:~/Documents/automated_testing/ui-testing$ npm test

> ui-testing@1.0.0 test
> jest

PASS ./ui.test.js (38.627 s)
  Vivat UI tests
    ✓ should open the Vivat home page (5485 ms)
    ✓ should display page title with Vivat information (6311 ms)
    ✓ should contain navigation links on the home page (6437 ms)
    ✓ should open books category page (5130 ms)
    ✓ should display book-related content on books category page (5094 ms)
    ✓ should have multiple links on books category page (5043 ms)
    ✓ should have visible page content (4853 ms)

Test Suites: 1 passed, 1 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        38.671 s
Ran all test suites.
• oksanka@debian:~/Documents/automated_testing/ui-testing$

```

Після успішного запуску тестів я підготувала проєкт до завантаження на GitHub. У мене раніше уже був ініціалізований Git-репозиторій для виконання цих робіт.

Тому першим ділом я перевірила стан файлів:

```

oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Невідстежувані файли:
  (скористайтесь "git add <файл>...", щоб додати до майбутнього коміту)
    ui-testing/

нічого не додано до коміту, але є невідстежувані файли (скористайтесь "git add" для відстежування)
oksanka@debian:~/Documents/automated_testing$

```

Далі я додала всі файли до індексації Git:

```

oksanka@debian:~/Documents/automated_testing$ git add .
oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Зміни, додані до майбутнього коміту:
  (використовуйте "git restore --staged <файл>...", щоб видалити з індексу)
    новий файл:   ui-testing/.gitignore
    новий файл:   ui-testing/package-lock.json
    новий файл:   ui-testing/package.json
    новий файл:   ui-testing/ui.test.js

oksanka@debian:~/Documents/automated_testing$

```

Після цього я створила коміт:

```

oksanka@debian:~/Documents/automated_testing$ git commit -m "Add UI tests for lab 5"
[main 302b672] Add UI tests for lab 5
4 files changed, 5302 insertions(+)
 create mode 100644 ui-testing/.gitignore
 create mode 100644 ui-testing/package-lock.json
 create mode 100644 ui-testing/package.json
 create mode 100644 ui-testing/ui.test.js
oksanka@debian:~/Documents/automated_testing$

```

Потім я завантажила проєкт на GitHub:

```
oksanka@debian:~/Documents/automated_testing$ git push -u origin main
Username for 'https://github.com': OksanaMykytyn
Password for 'https://OksanaMykytyn@github.com':
Перерахування об'єктів: 8, готово.
Підрахунок об'єктів: 100% (8/8), готово.
Дельта компресія з використанням до 4 потоків
Компресія об'єктів: 100% (6/6), готово.
Запис об'єктів: 100% (7/7), 43.56 KiB | 8.71 MiB/c, готово.
Всього 7 (дельта 1), повторно використано 4 (дельта 0), повторно використано пакунків 0
(з 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/OksanaMykytyn/automated_testing.git
   0f8bd88..302b672  main -> main
Гілку "main" налаштовано на відстежування "origin/main".
oksanka@debian:~/Documents/automated_testing$ |
```

Посилання на GitHub:

https://github.com/OksanaMykytyn/automated_testing

Висновок: під час лабораторної роботи я навчилася створювати автоматизовані UI-тести за допомогою Jest та Puppeteer. Я обрала для перевірки сайт видавництва Віват, оскільки він має зрозумілий інтерфейс, каталог книжок, навігацію та сторінки з контентом. Я налаштувала Node.js-проєкт, встановила Jest і Puppeteer та створила файл ui.test.js для тестів. Під час виконання роботи я написала тести для перевірки відкриття головної сторінки, заголовка сторінки, наявності посилань, відкриття категорії книг і відображення текстового вмісту. Я використала page.goto, page.title, page.url, \$eval та \$\$eval для взаємодії зі сторінкою та перевірки її елементів.